

Change Data Capture on Google Cloud

Datastream + Landing Zone Patterns

A practical, beginner-friendly handbook: what CDC is, how Datastream captures changes from a MySQL database, how to design a landing zone in Cloud Storage and BigQuery, and how to build, run, and troubleshoot the whole pipeline yourself.

Companion asset	cdc_datastream_landing_zone_demo.ipynb (Google Colab notebook)
Primary services	Datastream · Cloud SQL for MySQL · Cloud Storage · BigQuery
Audience	Data engineers new to CDC and Datastream; trainers running a live demo
Demo footprint	Smallest practical Cloud SQL tier (db-f1-micro / db-perf-optimized-N-2), single table

Contents

- | | |
|----|--|
| 1 | What Problem Does CDC Actually Solve? |
| 2 | Core Terminology, Explained Simply |
| 3 | How Datastream Captures Changes |
| 4 | The Landing Zone Pattern |
| 5 | Reference Architecture |
| 6 | Step-by-Step Build Walkthrough |
| 7 | Schema Evolution & Data Type Handling |
| 8 | Operations: Monitoring, Cost, Security, Sizing |
| 9 | Best-Practice Checklist |
| 10 | Troubleshooting Guide |
| 11 | Appendix: Full Glossary & Further Reading |

1. What Problem Does CDC Actually Solve?

Most operational systems — the checkout service, the CRM, the billing database — store their current state in a transactional database such as MySQL or PostgreSQL. Analytics, machine learning, and reporting systems need that same data, but they cannot simply query the production database directly: every extra query load on it risks slowing down the application that pays the bills.

The classic answer was **batch ETL**: once a night, run a query that pulls the whole table (or rows changed since yesterday, based on an *updated_at* column), and load it into a warehouse. This works, but it has three structural weaknesses that **Change Data Capture (CDC)** is designed to remove:

- **Freshness** — nightly batches mean the warehouse is always at least a day stale. Fraud detection, inventory, and personalization use cases often can't tolerate that.
- **Deletes are invisible** — a query for "rows changed since yesterday" cannot see a row that was deleted, because it's gone. Batch ETL routinely loses deletes unless it does expensive full reconciliation.
- **Load on the source** — a full nightly scan of a multi-million-row table competes for the same disk I/O and CPU as live customer traffic, right when analytics teams want the freshest possible snapshot.

CDC solves all three by reading the database's own **transaction log** (MySQL's binary log, or "binlog") rather than querying tables. Every insert, update, and delete the database engine commits is already written to this log for its own replication and crash-recovery purposes. CDC tools tap into that stream, so capturing a change costs the source almost nothing beyond serving the log itself, and deletes appear as a normal event in the log just like every other change.

Why this matters for training

This is the single idea to anchor every later section on: **CDC does not query your tables — it tails your database's transaction log**. Everything about Datastream's behavior (why it needs binlog access, why it does an initial "backfill", why schema changes are tricky) follows from this one fact.

Google Cloud's managed CDC service for this job is **Datastream**. It connects to a source database (MySQL, PostgreSQL, Oracle, or SQL Server), performs a one-time backfill of existing data, then streams every subsequent change, in order, to a destination — typically Cloud Storage or BigQuery. It is serverless: there is no cluster to size or patch, and Google manages scaling and high availability.

2. Core Terminology, Explained Simply

Read this section slowly if you are new to CDC — nearly every later section reuses these words without re-explaining them.

Change Data Capture (CDC)

A technique for detecting and delivering row-level inserts, updates, and deletes as they happen in a source database, typically by reading the database's internal transaction log rather than querying its tables.

Binary log (binlog)

MySQL's append-only log of every data-changing statement or row event, originally built so replica databases can copy changes from a primary. Datastream reads this log the same way a MySQL replica would — it registers itself as a replication client.

GTID (Global Transaction Identifier)

A unique ID MySQL assigns to every committed transaction when GTID mode is enabled. Datastream (and any replication consumer) uses GTIDs or binlog file+offset positions to track exactly how far it has read, so it can resume precisely after a restart without re-reading or skipping events.

Backfill

The one-time bulk copy of a table's existing rows, performed when a stream first starts (or when a new table is added to a stream). Datastream runs the backfill and the live change stream together, so no changes are lost in between — rows changed after the backfill query ran are captured from the log and reconciled into the destination.

Connection profile

A reusable Datastream object that stores how to reach one system — the hostname/IP, port, credentials, and network path (public IP, forward-SSH-tunnel, or private connectivity). You create one connection profile for the source (MySQL) and one for the destination (Cloud Storage or BigQuery).

Stream

The Datastream object that ties a source connection profile to a destination connection profile, plus the list of schemas/tables to include, and the destination output format. Starting a stream kicks off backfill + ongoing CDC.

Private connectivity configuration

A Datastream resource that peers Datastream's Google-managed network into your VPC, so it can reach a Cloud SQL instance (or any database) on a private IP without exposing it to the public internet.

Landing zone

The first storage location where raw captured data lands, before any transformation. In this handbook it's a Cloud Storage bucket (or a raw BigQuery dataset) that holds an unmodified, append-only record of every change event, structured for replay and audit.

Bronze / Silver / Gold (medallion architecture)

A common naming convention for progressively refined data layers: **Bronze** (raw landing zone, as captured, one row per change event, includes deletes), **Silver** (deduplicated, merged into current-state tables that mirror the source schema), **Gold** (business-level aggregates and marts built for specific reporting or ML use cases).

Merge / Upsert

The operation that reconciles a stream of insert/update/delete events into a single current-state row per primary key — inserting new keys, overwriting existing keys with the latest version, and removing keys that were deleted. BigQuery's Datastream integration runs this automatically as a background MERGE job.

Unified types

Datastream converts each source database's native column types (e.g. MySQL's TINYINT, DATETIME) into a common, database-agnostic type system before writing output, so the same downstream code can handle CDC data from MySQL, PostgreSQL, or Oracle sources consistently.

Idempotency / at-least-once delivery

Datastream guarantees each change event is delivered at least once, meaning a rare retry could deliver the same event twice. Destination pipelines must be safe to apply the same event more than once, usually by keying merges on the primary key plus the event's log position/timestamp.

Schema drift

Any change to a source table's structure — adding a column, widening a type, dropping a column — that happens after a stream is already running. Datastream detects most DDL changes automatically and propagates them, but some (like a dropped and renamed column) need manual handling; see Section 7.

3. How Datastream Captures Changes

3.1 The two phases of every stream

Every Datastream stream runs two things concurrently from the moment you click **Start**:

- 1 **Backfill** — Datastream issues read queries against the source tables you selected (in manageable chunks, to avoid overloading the source) and writes the current row values to the destination as a full load.
- 2 **CDC via the transaction log** — at the same time, Datastream begins reading the binlog from the position it was at when the stream started, so any row changed *during* the backfill is captured too. Datastream reconciles ordering so the destination ends up correct even though these two processes run in parallel.

3.2 Getting Datastream access to the binlog

For a MySQL source, three things must be true before Datastream can start:

- **Binary logging is enabled** and set to ROW format (Cloud SQL for MySQL enables this automatically when you turn on point-in-time recovery, which Datastream requires).
- **A database user exists with replication privileges** — REPLICATION SLAVE, REPLICATION CLIENT, and SELECT on the schemas being captured.
- **Network connectivity** from Datastream's infrastructure to the database's IP and port — see 3.3 below.

3.3 Three ways Datastream reaches your database

Method	How it works	When to use it
Public IP + IP allowlist	Database exposes a public IP; Datastream's published IP range is added to the database's authorized networks.	Quick demos and training labs where the source has no sensitive production traffic.
Forward SSH tunnel	Datastream connects out to a small bastion VM you control over SSH; the bastion forwards traffic to the private database.	Database stays fully private; no VPC peering setup required.
Private connectivity (VPC peering)	A dedicated Datastream private connectivity configuration peers directly into your VPC.	Production pipelines, and any Cloud SQL instance that only has a private IP.

Demo note

The companion notebook uses the **public IP + authorized network** method because it needs zero extra VPC/bastion setup and is the fastest path to a working demo. Section 8.3 explains how to switch to private connectivity for anything beyond a lab.

3.4 Where the data goes

Datastream writes to one of two destination types:

- **Cloud Storage (the landing zone)** — writes self-describing Avro or JSON files, one folder per table, partitioned by time, containing raw change events (including a special `_metadata_change_type` column: INSERT/UPDATE-INSERT/UPDATE-DELETE/DELETE). This is the pattern this handbook focuses on.
- **BigQuery (direct destination)** — Datastream writes change events into a staging table and runs a continuous background MERGE to maintain a queryable, current-state replica table automatically, without you writing any merge SQL yourself.

4. The Landing Zone Pattern

A "landing zone" is deliberately the dumbest, most faithful layer in the pipeline: it should hold exactly what Datastream produced, unmodified, so it can be replayed if a downstream job has a bug or a schema needs to be reprocessed from scratch. Everything clever — deduplication, merging, business logic — happens in layers built *on top of* the landing zone, never inside it.

4.1 Why land in object storage at all, instead of writing straight to BigQuery?

- **Cheapest possible durability.** Cloud Storage is far cheaper per GB than keeping years of raw change history in BigQuery active storage.
- **Replayability.** If a downstream transformation has a bug, you can reprocess from the raw files without re-extracting from the source database (which may have already deleted the rows in question).
- **Decoupling.** Multiple downstream consumers — BigQuery, Dataflow jobs, even a different warehouse — can all read the same landing zone independently.
- **Format flexibility.** Avro files carry an embedded schema, which makes schema evolution and cross-tool compatibility easier than a live database connection.

The trade-off is that object storage is not queryable the way a table is — something has to load or merge it into BigQuery for analysts to use it. That's the job of the next layer.

4.2 A three-layer landing pattern

Layer	Contents	Update pattern
Raw / Bronze (Cloud Storage)	Every change event Datastream wrote — inserts, updates, deletes — as Avro/JSON files, partitioned by table and by ingestion timestamp.	Append-only. Never edited or overwritten.
Staged / Silver (BigQuery)	One current-state table per source table, primary-key deduplicated. Either Datastream's built-in BigQuery merge, or a scheduled query/Dataflow job that reads the GCS files and MERGES into BigQuery.	Continuously or periodically merged/upserted.
Curated / Gold (BigQuery views or tables)	Business-ready marts: joins across tables, aggregates, slowly changing dimensions, ML feature tables.	Scheduled transformation jobs (dbt, Dataflow, scheduled queries).

4.3 Suggested object layout in the landing bucket

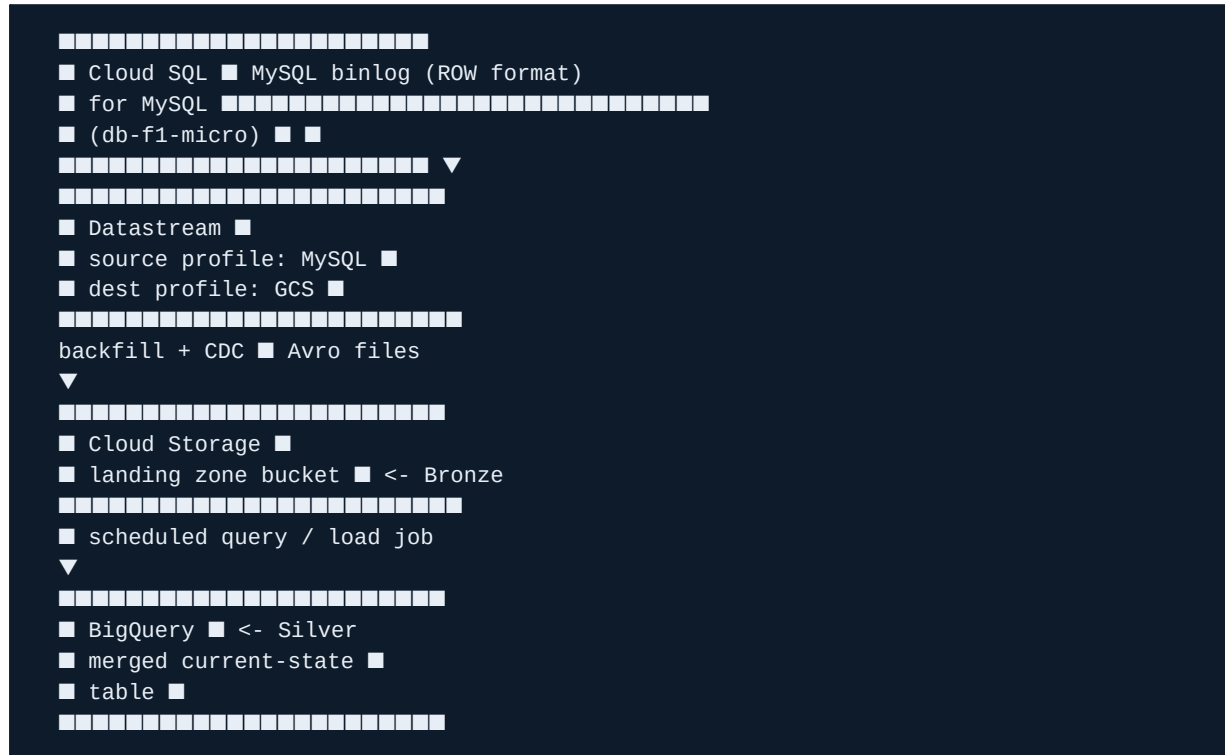

```
gs://<landing-bucket>/  
<stream-name>/  
<database>/  
<schema>/  
<table>/  
2026/07/16/14/  
<uuid>.avro  
<uuid>.avro  
  
# Datastream partitions automatically by year/month/day/hour of write time,  
# which keeps individual objects small and makes lifecycle/retention rules trivial  
# to write per-table.
```

Tip

Set a Cloud Storage **lifecycle rule** on the landing bucket (e.g. move objects to Nearline after 30 days, delete after 1 year) once you have a downstream layer that has already consumed and merged the data — the landing zone doesn't need to be the permanent home for history once BigQuery holds the merged state.

5. Reference Architecture

The diagram below is the pattern this handbook (and the companion notebook) implements end to end, using the smallest practical footprint for a training demo.



5.1 Component responsibilities

Component	Responsibility
Cloud SQL for MySQL	The operational source database. Binary logging + a replication user are the only special requirements.
Datastream connection profile (source)	Stores how to reach Cloud SQL: host, port, credentials, network path.
Datastream connection profile (destination)	Stores the target GCS bucket/path or BigQuery dataset.
Datastream stream	Ties the two profiles together, selects which tables to capture, and runs backfill + CDC.
Cloud Storage bucket	The landing zone — raw, replayable change history in Avro.
BigQuery dataset	Holds the merged, queryable current-state table(s) used for analytics.

Design choice

This reference architecture lands in **GCS first, then BigQuery** so you can see and inspect the raw change events directly (great for teaching). In production, many teams point Datastream straight at BigQuery as the destination and let Datastream's built-in merge maintain the current-state table, skipping a manual GCS→BigQuery load step. Both patterns are covered in the notebook.

6. Step-by-Step Build Walkthrough

This section narrates exactly what the companion notebook automates, so you understand every step even if you run it as one click. Each step below is numbered to match a section in the notebook.

Step 1 — Enable the required APIs

```
gcloud services enable \
  sqladmin.googleapis.com \
  datastream.googleapis.com \
  storage.googleapis.com \
  bigquery.googleapis.com \
  --project=$PROJECT_ID
```

Step 2 — Create the smallest usable Cloud SQL for MySQL instance

For a demo, we don't need high availability or a large tier. db-f1-micro is the smallest shared-core tier available for Cloud SQL and is sufficient to hold a small demo table and generate a handful of CDC events.

```
gcloud sql instances create cdc-demo-mysql \
  --database-version=MYSQL_8_0 \
  --tier=db-f1-micro \
  --region=us-central1 \
  --storage-size=10 \
  --storage-type=SSD \
  --enable-bin-log \
  --backup-start-time=03:00
```

Why --enable-bin-log matters

Cloud SQL does not turn on binary logging by default. Datastream cannot read the binlog — and therefore cannot capture changes — unless this flag (or the equivalent Console toggle under Data Protection → Point-in-time recovery) is set *before* the stream is created.

Step 3 — Create a replication-only database user for Datastream

```
gcloud sql users create datastream_user \
  --instance=cdc-demo-mysql \
  --password=$DATASTREAM_PASSWORD

# Then, connected as root, grant the minimum privileges Datastream needs:
GRANT REPLICATION SLAVE, REPLICATION CLIENT, SELECT
ON *.* TO 'datastream_user'@'%';
FLUSH PRIVILEGES;
```

Step 4 — Seed a small demo table

```
CREATE DATABASE demo;
USE demo;
CREATE TABLE orders (
  order_id INT PRIMARY KEY AUTO_INCREMENT,
  customer_name VARCHAR(100),
  amount DECIMAL(10,2),
  status VARCHAR(20),
  updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
  ON UPDATE CURRENT_TIMESTAMP
);
INSERT INTO orders (customer_name, amount, status) VALUES
('Asha Rao', 1499.00, 'PLACED'),
('Vikram Iyer', 799.50, 'PLACED');
```

Why a primary key matters

Datastream (and BigQuery's downstream merge) needs a primary key to know which rows to update or delete during merges. A table with no primary key can still be captured, but the destination will only ever be append-only — there is no reliable key to merge on.

Step 5 — Allow Datastream to reach the instance (public IP path)

For the training-lab network path, add Cloud SQL's public IP and authorize Datastream's published IP ranges as an authorized network:

```
gcloud sql instances patch cdc-demo-mysql \
--assign-ip \
--authorized-networks=<Datastream's published IP range for your region>
```

Look up the real IP range

Datastream's per-region public IP addresses are published in the official documentation and can change — always look up the current value for your region rather than hardcoding an old one. The notebook fetches and prints this for you.

Step 6 — Create the destination landing bucket

```
gcloud storage buckets create gs://$PROJECT_ID-cdc-landing \
--location=us-central1 \
--uniform-bucket-level-access
```

Step 7 — Create the two Datastream connection profiles

```
gcloud datastream connection-profiles create mysql-source-profile \
--location=us-central1 \
--type=mysql \
--mysql-hostname=<Cloud SQL public IP> \
--mysql-port=3306 \
--mysql-username=datastream_user \
--mysql-password=$DATASTREAM_PASSWORD

gcloud datastream connection-profiles create gcs-landing-profile \
--location=us-central1 \
--type=google-cloud-storage \
--bucket=$PROJECT_ID-cdc-landing \
--root-path=/cdc-demo
```

Step 8 — Create and start the stream

```
gcloud datastream streams create orders-cdc-stream \
--location=us-central1 \
--source=mysql-source-profile \
--mysql-source-config='{"includeObjects":{"mysqlDatabases":[{"database":"demo"},
"mysqlTables":[{"table":"orders"}]}}' \
--destination=gcs-landing-profile \
--gcs-destination-config='{"avroFileFormat":{}}' \
--backfill-all

gcloud datastream streams update orders-cdc-stream \
--location=us-central1 --state=RUNNING --update-mask=state
```

Step 9 — Generate live changes and watch them land

With the stream running, insert, update, and delete rows in orders and watch new Avro files appear under `gs://$PROJECT_ID-cdc-landing/cdc-demo/...` within roughly seconds to a couple of minutes.

Step 10 — Read the landing zone and merge into BigQuery

The notebook reads the Avro files with `pandas + fastavro`, shows the raw change events (including the `_metadata_change_type` column), then loads them into a BigQuery staging table and runs a `MERGE` to produce a deduplicated current-state table — demonstrating the Silver layer by hand so the mechanics are visible, even though Datastream can also do this automatically with a BigQuery destination.

7. Schema Evolution & Data Type Handling

7.1 What Datastream handles automatically

- **Adding a column** — new columns appear in subsequent change events; Datastream updates the Avro schema and (for BigQuery destinations) adds the column to the destination table automatically.
- **Widening a compatible type** — e.g. INT to BIGINT — is generally handled without manual intervention.

7.2 What usually needs manual attention

- **Dropping a column** — historical events in the landing zone still contain the old column; downstream consumers need to tolerate columns that stop appearing.
- **Renaming a column** — MySQL's binlog sees this as drop + add, not a rename, so Datastream (and your merge logic) will see it as a new column with historical null values, not a seamless rename.
- **Changing a primary key** — because merges are keyed on the primary key, changing it requires re-backfilling the table into a new destination table, not an in-place migration.
- **Incompatible type narrowing** — e.g. VARCHAR(255) to VARCHAR(50) on a column that has longer historical values already captured.

Practical rule of thumb

Treat the landing zone's Avro schema as append-only history. Never try to force old and new schema versions to look identical — instead, have the Silver-layer merge job read each Avro file's own embedded schema and apply defaults/casts explicitly for any column that isn't present.

8. Operations: Monitoring, Cost, Security, Sizing

8.1 Monitoring a stream

- The Datastream Console page for a stream shows **backfill progress per table** and **current replication lag** (how far behind the live binlog position the stream is).
- Cloud Monitoring exposes Datastream metrics — `total_latency`, `freshness`, and per-object error counts — that can be wired into alerting policies.
- A stream in an **error state** almost always means either a network/connectivity break to the source, or the source's binlog retention expired before Datastream could read a segment (see 8.4).

8.2 Cost shape

Cost driver	Notes
Datastream	Billed per GB of data processed (streamed + backfilled), not per hour — an idle stream with no source changes costs very little.
Cloud SQL	db-f1-micro is billed hourly for compute plus storage; stop or delete the instance after a training session to avoid ongoing charges.
Cloud Storage	Landing zone storage is cheap per GB; apply lifecycle rules once data has been merged downstream.
BigQuery	Storage for the merged tables plus query costs; Datastream's automatic merge into BigQuery also consumes BigQuery slot/query cost.

8.3 Moving from public IP to private connectivity

For anything beyond a lab, replace the public-IP + authorized-networks setup with a Datastream **private connectivity configuration**: this creates a VPC peering connection between Datastream's Google-managed network and your VPC, so Cloud SQL never needs a public IP at all.

```
gcloud datastream private-connections create cdc-private-conn \
  --location=us-central1 \
  --display-name="CDC demo private connection" \
  --vpc=projects/$PROJECT_ID/global/networks/default \
  --subnet=10.10.10.0/29
```

8.4 Binlog retention — a common failure mode

Warning

If Datastream is paused, or falls behind, for longer than your MySQL binlog retention window (Cloud SQL default is often short, e.g. a few hours to a day depending on configuration), the log segments it needs may be purged before it can read them, forcing a full re-backfill. For anything but a short demo, increase `binlog_expire_logs_seconds` via a Cloud SQL database flag to a safety margin larger than your expected maximum pause.

8.5 Sizing guidance

Scenario	Suggested Cloud SQL tier
Training / demo, single small table	db-f1-micro (this handbook's default)
Light dev/test workload, a few tables	db-g1-small or db-custom-1-3840
Production source feeding CDC	Sized for the production workload itself — CDC read load via the binlog is a minor addition, not the driver of sizing.

9. Best-Practice Checklist

- 1 Every captured table has a primary key.
- 2 Binary logging is enabled with ROW format before the stream is created, not after.
- 3 The Datastream database user has only the privileges it needs (replication + SELECT), not broader admin rights.
- 4 Binlog retention comfortably exceeds your expected maximum stream downtime.
- 5 The landing zone bucket is treated as append-only and never manually edited.
- 6 A lifecycle policy controls how long raw landing-zone data is retained once it has been merged downstream.
- 7 Downstream merge logic is idempotent — safe to re-apply the same change event more than once.
- 8 Monitoring is wired to alert on stream errors and rising replication lag, not just "stream is not running".
- 9 For anything beyond a lab, private connectivity replaces public-IP + authorized networks.
- 10 Schema-changing DDL on the source has an agreed process (announce, coordinate, verify Datastream propagated it) rather than being silently applied.

10. Troubleshooting Guide

Symptom	Likely cause	What to check
Stream stuck at 0% backfill	Network path to the source isn't open yet.	Authorized networks / firewall rules / private connectivity peering status.
Stream shows a connectivity error on the source profile	Wrong credentials, or binlog not enabled.	Re-test the connection profile; confirm <code>--enable-bin-log</code> and the replication user's grants.
Stream running but no new files appear after inserting rows	Change hasn't been flushed/committed, or file rotation interval hasn't elapsed.	Confirm the transaction committed; Datastream batches writes, so allow a short delay.
Backfill restarts from scratch unexpectedly	Binlog position was lost — often due to binlog retention expiring during a long pause.	Increase <code>binlog_expire_logs_seconds</code> ; avoid long stream pauses.
BigQuery destination table missing rows present in source	Primary key missing on the source table, so merge can't identify rows.	Add a primary key on the source table (requires a schema change + re-backfill).
Duplicate rows in a hand-rolled merge (GCS -> BigQuery)	Merge logic isn't deduplicating on primary key + latest change timestamp.	Use <code>ROW_NUMBER()</code> partitioned by primary key, ordered by the log position/timestamp, keeping only the latest.

11. Appendix: Full Glossary & Further Reading

Full glossary (quick reference)

Term	Plain-English meaning
CDC	Change Data Capture — detecting and delivering row-level changes from a source system.
Binlog	MySQL's transaction log, originally for replication, that CDC tools read.
GTID	Global Transaction Identifier — a unique ID per committed MySQL transaction.
Backfill	One-time bulk copy of existing rows when a stream (or a new table) starts.
Connection profile	Datastream object storing how to reach one system (source or destination).
Stream	Datastream object linking a source and destination profile plus the tables to capture.
Private connectivity	VPC peering that lets Datastream reach a private-IP database.
Landing zone	First, raw, append-only storage location for captured change data.
Bronze/Silver/Gold	Medallion naming for raw, deduplicated, and business-ready data layers.
Merge/Upsert	Reconciling a change stream into one current-state row per primary key.
Unified types	Datastream's database-agnostic column type system.
Idempotency	Downstream logic being safe to re-apply the same event more than once.
Schema drift	Any source table structure change occurring after a stream is already running.
Replication lag / freshness	How far behind the live source the stream's applied changes are.

Where to go next

- Google Cloud Datastream documentation — official docs, always the source of truth for current IP ranges, quotas, and supported source/destination combinations.
- Cloud SQL for MySQL documentation — binary logging, point-in-time recovery, and database flags.
- BigQuery documentation on Datastream destinations — how the automatic staging + merge works.
- The companion notebook, `cdc_datastream_landing_zone_demo.ipynb`, which runs every step in this handbook against a real (small) GCP project.

End of handbook. Pair this with the companion Colab notebook for a hands-on walkthrough.